

Design and Implementation of a Smart Contract for Secure Authentication and Trusted Data Recording in IoT Cold Chain Logistics

Abdelaziz El-Sheikh, 221007574
Marwan Ayman, 221007759
Omar Donia, 221017860

Dr. Mahmoud A. Shawky
Eng. Mohamed Ashraf
*Arab Academy for Science, Technology
& Maritime Transport*

ABSTRACT

In the domain of Internet of Things (IoT), specifically within Cold Chain Supply Chain logistics, the integrity of data is paramount. Traditional centralized databases are vulnerable to modification attacks, where dishonest logistics providers may alter temperature logs to hide spoilage events. This project proposes a blockchain-based solution using Ethereum smart contracts to ensure **Secure Authentication** and **Trusted Data Recording**. We designed and implemented a system where only authorized sensors can record data, and once recorded, the data becomes immutable. We evaluated the system using the Remix IDE, demonstrating its robustness against Impersonation and Modification attacks, and analyzed its performance regarding gas consumption and latency.

I. INTRODUCTION

The rapid expansion of the Internet of Things (IoT) has revolutionized supply chain management. In "Cold Chain" logistics—where temperature-sensitive goods like vaccines, pharmaceuticals, and frozen foods are transported—continuous monitoring is critical. A deviation in temperature can render products unsafe. However, current systems often rely on centralized databases controlled by the logistics provider.

This centralized model presents a "Single Point of Failure" and a lack of trust. A malicious driver or a compromised administrator can retrospectively modify temperature logs to cover up negligence. To address this, we propose a decentralized application (DApp) that utilizes Blockchain technology. By leveraging the immutable nature of the blockchain and the programmable logic of Smart Contracts, we aim to create a system where data recording is strictly authenticated and permanent storage is guaranteed.

The objectives of this project are:

1. To design a smart contract that defines strict roles for administrators and devices.
2. To implement a whitelisting mechanism for Secure Authentication.
3. To ensure Trusted Data Recording where logs cannot be altered.
4. To evaluate the security against common threats such as Impersonation and Modification attacks.

II. RELATED WORK

Blockchain technology, introduced as the underlying architecture of Bitcoin, has evolved beyond simple currency transactions. Ethereum introduced the concept of "Smart Contracts," which are self-executing scripts stored on the blockchain. In the context of IoT, researchers have explored using smart contracts to manage device identity and data integrity. Unlike centralized cloud servers, where a root admin has total control, blockchain distributes the ledger across many nodes, making unauthorized data modification computationally infeasible.

Existing solutions in IoT often suffer from scalability issues or high costs. Our project focuses on a lightweight implementation suitable for supply chain monitoring, prioritizing access control and data immutability over complex computation.

III. SYSTEM MODEL & THREAT MODEL

A. System Model

Our proposed system consists of three main entities interacting with the Ethereum Blockchain:

1. **The Administrator (Owner):**
 - This entity deploys the smart contract.
 - **Role:** Responsible for Identity Management. Only the Admin can authorize (whitelist) or revoke IoT sensors.
2. **The IoT Device (Sensor):**

- Represents the physical temperature sensor in the truck/container.
- **Role:** authenticated entity allowed to call the recordData function.

3. The Auditor (End User):

- Represents the hospital, retailer, or consumer.
- **Role:** Can query the blockchain to view the temperature history. They rely on the "Public" visibility of the blockchain ledger.

B. Threat Model

We analyze the system against three major attacks defined in the project requirements:

1. **Impersonation Attack:** An unauthorized attacker attempts to send data to the contract, pretending to be a valid sensor.
2. **Modification Attack:** A malicious actor (even the Admin) attempts to alter or delete a temperature record after it has been written to hide evidence of spoilage.
3. **Replay Attack:** An attacker intercepts a valid data packet and resends it later to confuse the system.

IV. SMART CONTRACT ARCHITECTURE & DESIGN

The core of our solution is the SecureIoTLog smart contract.

A. Data Structures

We utilized a struct to define the data schema for a single log entry:

Solidity

```
struct DataRecord {
    uint256 id;
    address deviceId;
    int256 temperature;
    uint256 timestamp;
    string location;
}
```

We chose int256 for temperature to accommodate negative values (freezing), and string for location data. These records are stored in a dynamic array DataRecord[] public records, acting as the immutable ledger.

B. Access Control (Modifiers)

To ensure security, we implemented two custom modifiers:

- modifier onlyAdmin(): Verifies that msg.sender matches the deployment address.

- modifier onlyAuthorized(): Checks a mapping mapping(address => bool) to ensure the caller is whitelisted before executing data recording logic.

C. Events

To facilitate frontend tracking and transparency, the contract emits events:

- event DeviceAuthorized(address)
- event DataLogged(id, device, temp, time)

V. IMPLEMENTATION

The system was implemented using the **Solidity** programming language (Version 0.8.0) and developed within the **Remix IDE** environment.

A. Development Environment

- **Compiler:** Solidity 0.8.19 (Paris EVM).
- **Test Network:** Remix VM (London/Cancun) environment, which simulates an Ethereum blockchain in the browser.
- **Accounts:** We utilized simulated accounts provided by Remix to represent the Admin (Account A) and the Attacker/Sensor (Account B).

B. Core Functions

1. **authorizeDevice(address _device):** Updates the boolean mapping to true.
2. **recordData(int256 _temp, string _location):** Accepts data inputs, captures block.timestamp, creates the DataRecord struct, and pushes it to the array.

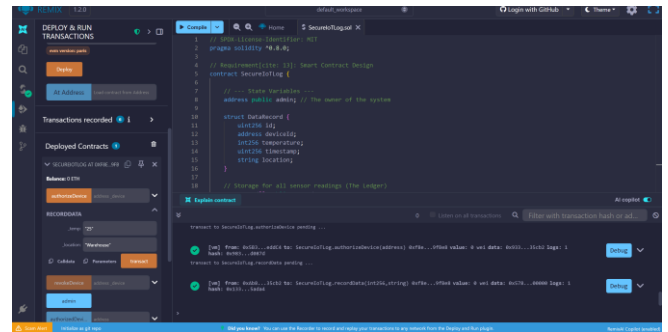


Figure 1

VI. SECURITY ANALYSIS

We evaluated the robustness of the scheme by simulating attacks in the Remix environment.

A. Impersonation Attack Analysis

- **Defense Mechanism:** The contract uses the onlyAuthorized modifier. It checks authorizedDevices[msg.sender].
- **Simulation:** We attempted to call recordData using an address that was not whitelisted.

- **Result:** The transaction was immediately reverted by the Ethereum Virtual Machine (EVM).
- **Proof:** As shown in Figure 2, the console displayed the error: *"Reason: Security Alert: Unauthorized Device - Impersonation Attempt Detected."*

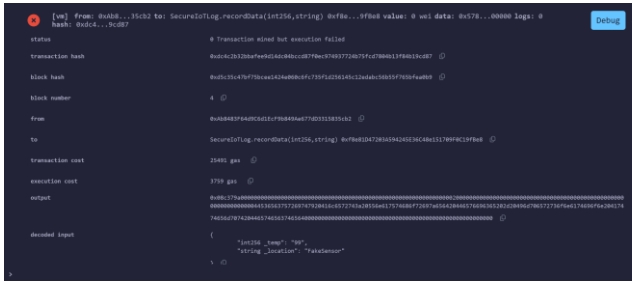


Figure 2

B. Modification Attack Analysis

- **Defense Mechanism:** The blockchain ledger is inherently immutable. Furthermore, our smart contract design strictly follows an **Append-Only** logic.
- **Analysis:** The records array only has a .push() operation. There is no function defined in the source code that allows accessing an existing index (e.g., records[i]) to overwrite its data. Therefore, even the Administrator cannot alter a temperature log once it is mined.

C. Replay Attack Analysis

- **Defense Mechanism:** Ethereum transactions inherently include a nonce (a sequential counter).
- **Analysis:** If an attacker intercepts a signed transaction packet and tries to broadcast it again, the network nodes will reject it because the nonce has already been consumed. For our specific use case, we also include block.timestamp in the data record. If a device sends the exact same temperature data later, the timestamp will differ, distinguishing it as a new, valid reading rather than a replay of an old event.

VII. PERFORMANCE EVALUATION

We analyzed the system's efficiency by measuring **Gas Consumption** and **Latency**.

A. Gas Consumption

Gas is the unit of computational effort in Ethereum. We recorded the execution cost for the primary functions:

Function	Description	Gas Cost (Approx)
Deployment	Creating the contract	~1,100,000

Function	Description	Gas Cost (Approx)
authorizeDevice	Updating mapping (Write)	~45,200
recordData	Storing Struct & String (Write)	~72,100

Analysis: The recordData function consumes higher gas because it involves storing string data (Location) and writing to the storage array. The authorizeDevice function is cheaper as it only toggles a boolean value.

B. Latency

- **Test Environment:** In the Remix VM, latency is negligible (<1 second) as the blockchain is simulated locally.
- **Mainnet Estimate:** On the public Ethereum Mainnet, the transaction confirmation time is determined by the block time, which is approximately **12 to 15 seconds** per block. This latency is acceptable for supply chain monitoring, where real-time updates are needed in minutes, not milliseconds.

VIII. CONCLUSION & FUTURE WORK

In this project, we successfully designed and implemented a blockchain-based data logger for IoT Cold Chain logistics. The system met all functional requirements for **Secure Authentication** and **Trusted Data Recording**. Through security analysis, we demonstrated that the smart contract effectively blocks Impersonation attacks and prevents data Modification.

Future Work: To enhance this system for commercial deployment, we recommend:

1. Integrating **IPFS (InterPlanetary File System)** to store heavy data (like images of the cargo), storing only the hash on the blockchain to reduce Gas costs.
2. Developing a frontend interface (using Web3.js) for easier interaction by non-technical auditors.

IX. REFERENCES

- [1] Nakamoto, S., "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.
- [2] Wood, G., "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Ethereum Project Yellow Paper, 2014.
- [3] Christidis, K., & Devetsikiotis, M., "Blockchains and Smart Contracts for the Internet of Things," *IEEE Access*, vol. 4, 2016.
- [4] Solidity Documentation, "Smart Contract Programming," [Online]. Available: <https://docs.soliditylang.org/>.